SFWRENG / MECHTRON 3K04 Software Development

Today: Interface documentation

McMaster University | Engineering Computing & Software

October 22nd, 2026

Dr. Angela Zavaleta Bernuy

High-Level Software Engineering Concepts

- How to structure the software so the requirements can be implemented safely, clearly, and in a way regulators will accept.
- We are going to cover four main topics:

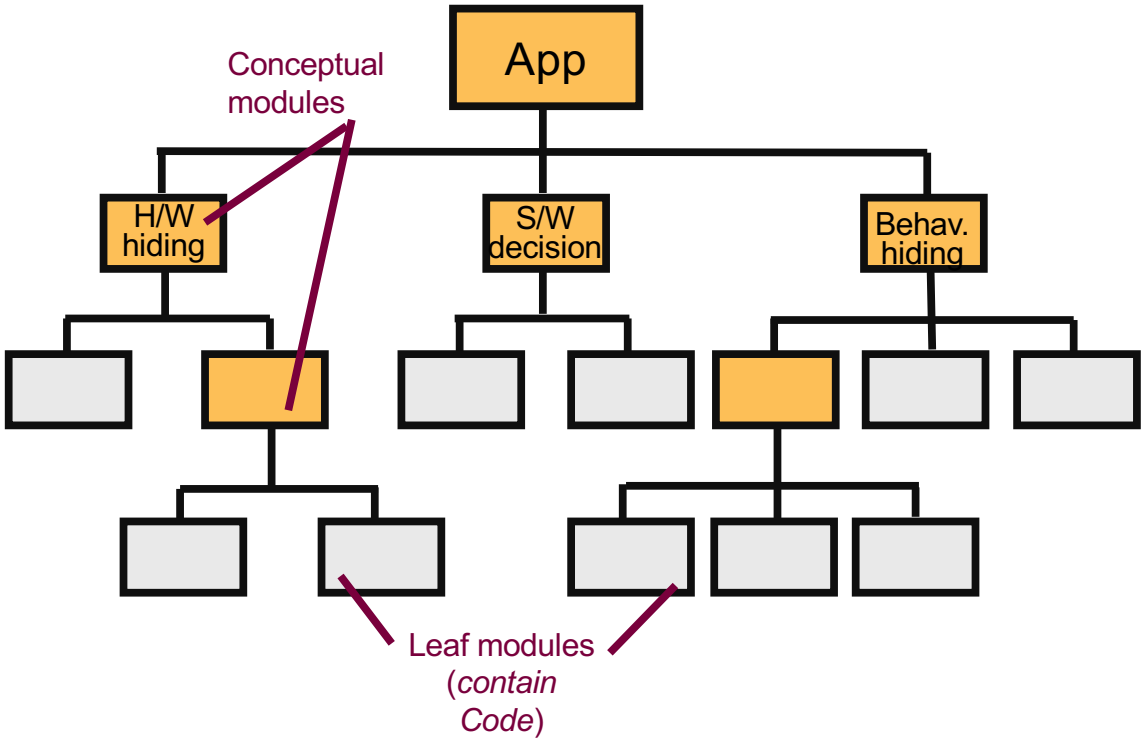
1. Relational Hierarchies
2. Cohesion and Coupling
3. Module Design ** **HERE** **
4. Module Documentation

3. Module Design: Design Frameworks/Notations

- Notations allow designs to be described precisely
- They can be be textual or graphical:
  - E.g., Parnas' Rational Design Process (RDP), Unified Modeling Language (UML), Z Notation, etc.
- We will expand one of these: Parnas' Rational Design Process (RDP)
  - It has 3 major components:
    - Module Guide (MG)
    - Module Interface Specification (MIS), and
    - Module Internal Design (MID)

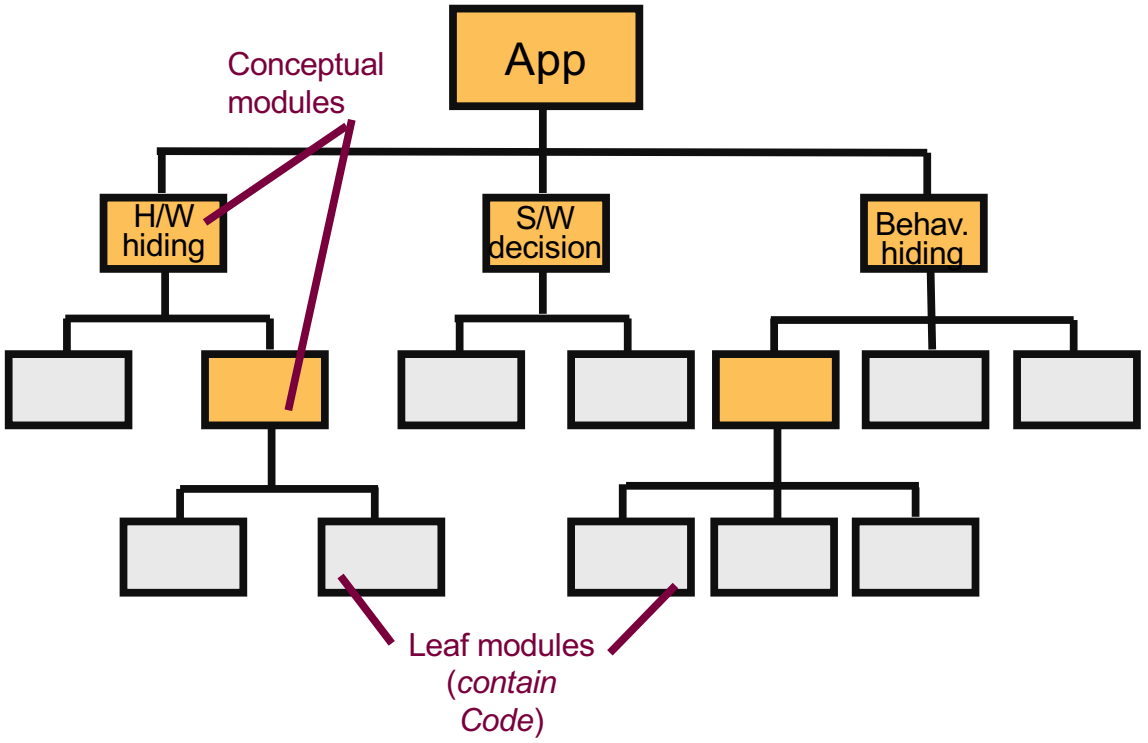
3. Module Design: RDP – MG

- When decomposing the system into modules (as per earlier discussions), we need to document the module decomposition so that the developers and other readers can understand the decomposition
- Parnas proposed a Module Guide (MG) based on the decomposition module tree shown earlier



3. Module Design: RDP – MG

- The MG consists of a table that documents each module's responsibility and *secret details*
- Conceptual modules will have broader responsibilities and *secret details*
- The leaf modules that represent code will contain much more precise responsibilities and *secret details*



3. Module Design: RDP – MG (Example)

#	Name	Responsibility	Secret Details
…	…	…	…
1.3	A/Is	Interface with the A/I hardware to produce software variables that represent the field data	Hardware representation of data & how to get data from hardware
1.3.1	FlowSensors	Software representation of flow measurements	Relationship between physical flow and software representation
…	…	…	…

Which one of these modules is conceptual and which one is a leaf?

3. Module Design: RDP – MG (Example)

ID	NAME	RESPONSIBILITY	SECRET
1	App	Entire system	All secrets
1.1	H/W hiding	All hardware related behaviour	List of h/w secrets
1.1.1	Analog input class 1	Behaviour for all analog I/O	Secret for class 1
1.1.2	Digital I/O	Behaviour for all digital I/O	List of secrets
1.1.2.1	Digital inputs class 2	Behaviour for all digital inputs	Secret for class 2
1.1.2.2	Digital outputs class 3	Behaviour for all digital outputs	Secret for class 3
1.2	S/W decisions	All behaviour for s/w decisions	List of secrets
1.2.1	S/W decision class 4	Behaviour for class 4	Secret for class 4
1.2.2	S/W decision class 5	Behaviour for class 5	Secret for class 5
1.3	Behaviour hiding	Most of the normal behaviour	List of secrets
1.3.1	…	…	…
1.3.2	Behaviour class 6	Behaviour for that class	Secret for class 6
1.3.3	Behaviour class 7	Behaviour for that class	Secret for class 7
1.3.1.1	Behaviour class 8	Behaviour for that class	Secret for class 8
1.3.1.2	Behaviour class 9	Behaviour for that class	Secret for class 9
1.3.1.3	Behaviour class 10	Behaviour for that class	Secret for class 10

3. Module Design: RDP – MIS

- For each leaf module, we need to document its interface and its implementation.
- In RDP, the interfaces are documented in the Module Interface Specifications (MIS)
- We would aim to use some form of mathematical approach to specify the MIS behaviour, as black-box as possible
- It describes what the interface to your class is, both syntax and semantics
  - Syntax: list of items with parameters
  - Semantics: description of behaviour of each method
    - For each method, describe the value of the outputs as they depend on the inputs

Notation and Documentation for the MIS

- The MIS can be documented in natural language or formally, but it is **key** that it is documented so that both *syntax* and *semantics* are specified
- Essentials:
  - Overview of services provided by the module
  - Syntax of the interface (exported types, constants and access programs)
  - Explanations of types and constants
  - For each access program, description of its black-box behaviour, not how it is implemented. Assumptions and limitations must also be described.
    - An excellent way of describing black-box behaviour of the access program is to describe the relation between outputs and inputs of the program.

What *Documenting and Interface* Means

For each module:

Field	What goes in it
Responsibility	One sentence: what this module is for
Secret	The anticipated change it hides (from I1)
Exported operations	Name, parameters with types, return type
Semantics	What each operation does, as a function table or pre/post pair
Assumptions	What the module takes as given (caller obligations, hardware behaviour)
Effects	What state changes, what gets logged, what the environment sees
Exceptions	What happens on invalid input or a failure, and what the caller is told

The rule from the iceberg slide applies: document **effects**, not only inputs and outputs. ("Returns a bool" is not semantics.)

Example: One module documented end to end

- **Module:** DoseChecker
- **Responsibility:** decide whether a programmed dose may be delivered
- **Secret:** how dose limits are represented and looked up

```
setLimits(drug: DrugId, soft: Rate, hard: Rate)
check(drug: DrugId, rate: Rate) -> Verdict
Verdict = { allowed, soft_override_required,
rejected }
```

- **Assumptions**: limits were loaded by pharmacy before any check; rate > 0 is the caller's obligation.
- **Effects**: none on delivery; every rejected verdict is logged with drug and rate.
- **Exceptions**: rate <= 0 returns rejected rather than throwing; the module never halts the system.

Semantics of check:

Condition	Verdict
drug not in library	`rejected`
`rate <= soft(drug)`	`allowed`
`soft(drug) < rate <= hard(drug)`	`soft_override_required`
`rate > hard(drug)`	`rejected`

The External Interface to the Partner System

Two teams, one boundary. Document it from your own side, in three parts:

- **Provides:** what your side offers the other (messages you send, requests you answer, with their meaning and timing).
- **Requires:** what you depend on from them, and what you assume about it (rate, ordering, acknowledgement).
- **Protocol is a secret:** the message format lives inside one comms module, so changing it touches nothing else.

Direction	Message	Meaning	Timing
Pump → Monitor	`STATUS(pumpId, state, rate, volume)`	Current operating state	Every 10 s
Pump → Monitor	`ALARM(pumpId, type, raisedAt)`	An alarm became active	Within 2 s of raising
Monitor → Pump	`ACK(pumpId, alarmId, userId)`	A nurse acknowledged it	On user action
Monitor → Pump	`QUERY(pumpId)`	Resend current status	On demand

Degraded Behaviour

How does your part behaves when the partner is unavailable? The rule for safety-critical systems: **the safety function never depends on the link.**

Failure	Pump side	Monitor side
Link down	Keeps infusing, alarms locally, buffers events	Marks the pump "communication lost" after 30 s, keeps last known state with its timestamp
Link restored	Sends buffered events in order	Replays them into the history, resolves stale flags
Partner never seen	Operates standalone; no safety function blocks	Shows the pump as absent, not as healthy
Duplicate or out-of-order message	Ignores duplicates by sequence number	Same

Mapping the Design onto Code

- One module = one file or one folder, named for the module in your Module Guide.
  - If a TA cannot find the boundary in a few seconds, the boundary is not real.
- The exported operations are the module's public methods. **Everything else is private.**
  - A public field is a leaked secret.
- Hardware-hiding modules get a real implementation and a simulated one behind the same interface.
  - E.g. that is how you demo an occlusion without breaking the hardware, and that is how you demo to your TAs!
- Keep the interface document and the code in the same order, with the same names.
- A short table in the design document, module → file → class, is what D2's implementation notes section is asking for.

Realizing a state machine in code

- States become an enum; the transition table from Week 4 becomes a lookup or a switch, not scattered ifs.
- Every event handler asks the table what the next state is.
  - Invalid event in the current state means "no change plus a log entry", exactly as specified.
- Guards from the diagram appear as the conditions on the transition, named the same way.
- Entry and exit actions live in one place per state
  - E.g. the alarm can never be left sounding by a path someone forgot.
  - If the TA asks "show me where this transition is", the answer is one line in one table.

Example: Extended Traceability

Req ID	Precise spec	Module	Code
FR-PRG-04	`check` (function table)	DoseChecker	`dose/DoseChecker.java`
FR-ALM-02	`Infusing → Alarm` on occlusion; invariant `alarmActive => motor stopped`	Alarm Manager, Sensor hiding	`alarm/AlarmManager.java`
NFR-PRF-02	`ALARM` sent within 2 s	Comms	`comms/Link.java`

Deliverable 2 Map

D2 Document Section	W5 L1	W5 L2	W5 L3	W6
1. Introduction and context	-	-	-	-
2. Precise specification	✓	✓		
3. Modular design and interfaces		✓	✓	
3.1. External ADC <-> Monitoring interface				✓
4. Implementation notes				✓
5. Traceability table				✓
6. Assumptions and decisions			✓	✓

D2 Demo (Nov 2th – Nov 6th)

- Have the system running before the TAs arrive, on the machine you present from.
- Be able to open the code and point at a module boundary, an interface, and a hidden secret.
- Be ready to trigger a normal scenario and an off-normal one on request.
- Rehearse the change drill: "the hospital switches to a different sensor”
  - I.e. which module changes, and what does not?
- Every member answers for every module. The TAs pick.

Questions?